Final Report: Intent Classification for Natural Language Queries

Course: EE541 A Computational Introduction to Deep Learning

Team Members: Huaitang Wang, Xiaotian Zheng

Date: 12.05.2025

1. Introduction

This project solves the task of intent classification from natural language, which determines 26 predefined travel-related intents (e.g., flight booking, fare inquiry, or service request). Formally, given a variable length of sentences $X = \{x_1, x_2, \ldots, x_n\}$, the objective is to predict a single intent label y from a set of $K = 26$ possible classes. Intent classification is a fundamental component of Natural Language Understanding in dialog systems, virtual assistants, and customer service automation. The accuracy of intent recognition directly determines whether a conversational agent can correctly route queries and execute appropriate actions. The ATIS (Air Travel Information System) dataset is the standard benchmark in this field, comprising spoken queries collected from users interacting with a flight booking system. This dataset has been widely adopted in both academic literature and industry applications for evaluating intent recognition systems.

The problem presents several challenges. Since we have a total of 26 intent labels, and many intent categories are really similar, the model we train must distinguish between labels with similar meanings. Class imbalance is also a problem when dealing with the dataset. The majority

X belong to a few common intents, resulting in other intents having few examples. Moreover, converting from the original dataset to a Neural-Network-readable dataset requires careful consideration. We must make sure each x_i is tokenized, embedded, and of equal length.

2. Related Work

Intent classification has evolved from traditional machine learning methods, such as Support Vector Machines and Naive Bayes classifiers, to deep learning approaches. More recently, CNN architectures, as proposed by Kim (2014) for sentence classification, have shown the effectiveness of feature extraction in intent detection. Later, better models like BERT(Devlin et al., 2018) use a method called self-attention to understand what each word means based on the words around it.

Our work follows the same path. We built a CNN model based on Kim's idea to see how well a good deep learning model can do. After that, we compare it with a more powerful BERT model. We also test a very simple MLP model to understand the importance of going deeper.

3. Dataset and Preprocessing

3.1 Data Loading and Initialization

We start by loading the ATIS dataset from two files: one for training and one for testing. The data was started in pickle format. Simply using the loading function in pickle, we get our train_data and test_data. After loading, we find there are 4978 training examples, 893 test examples, and 26 different types of intents. The data structure comes in a special format: each sentence is not stored as normal text but as numbers; for example, instead of seeing "want to fly", we see

something like [89, 242, 164]. There is also a dictionary that we can use to find the word from a number. We used the dictionary to turn each encoded sentence into a real-word sentence. After that, we get the result of sentences such as: “BOS i want to fly from boston at 838 am and arrive in denver at 1110 in the morning EOS”. The sentences are already lower-case, so we can process the data later.

3.2 Tokenization

There are four ways to do tokenization: 1. Word-level Tokenization, 2. Character-level Tokenization 3. Subword Tokenization 4. BERT Tokenization. In each way, we remove the BOS/EOS, since we want a clean dataset to support the model. For (1), Word-level is the most natural way for humans to think about language. We simply split each sentence into word units. The model can easily understand each word, but consequently, it will create a large vocabulary. For (2), we split sentences into individual letters and numbers. The advantage we get is that it creates a very small vocabulary, which is only 38 characters. However, for each sentence, the sequence becomes longer as usual. For (3), we use a heuristic suffix-based method. Each word is checked against a predefined suffix list and split into a stem and suffix when a matching rule is found. For (4), since the BERT model uses its own way of tokenization, we prepare that for later training of the model.

3.3 Vocabulary Construction and Padding

Since we cannot feed the words “flight” or “airfare" directly into a neural network, we need to convert them to numbers first. To build the vocabulary, we started by counting how many times each word appears in our training data. Words that appear only once might be spelling mistakes or very unusual words. Therefore, we decided to keep only words that appear at least twice, and

this helps our model focus on important words. We also added two special words: <PAD> for use to make all sentences the same length, and <UNK> for words we have never seen before.

Starting with our special words, we give every word a unique number; the common words get a lower number. For three different tokenizations, we build three different vocabularies. The word vocabulary has 631 entries, character vocabulary has 40 entries, and subword vocabulary has 593 entries. This is the corresponding effect we learn from the previous.

Natural sentences have different lengths, but neural networks need all input to be the same size to process them efficiently in batches. We analyze the statistics of the length of sentences, and find that 96% of sentences are 40 words or fewer. Based on this, we choose 40 words as our padding length and add <PAD> at the end of sentences if the length of the sentences is less than 40. There is a trade-off when we set the max length: when we cut long sentences, we will lose some information, but we will have efficient processing and short training times.

3.4 Final Dataset

Finally, we convert all the training data and the test data to an ID number and pad each sentence, then get the corresponding label for each sentence. We have the dataset size follow-up:

Method	Vocabulary size	Sentence length	Training samples	Test samples
Word	631	40	4978	893
Character	40	214	4978	893
Suffix-based subword	593	40	4978	893

BERT	pre-made	64	4978	893

3.5 Additional Dataset Variant and Data Augmentation

In addition to our primary ATIS dataset pipeline, we prepared a second dataset variant using word-level tokenization. All other preprocessing steps—lowercasing, sequence padding to a fixed MAX_LEN, and HDF5 serialization—were kept identical to ensure comparability across datasets.

On top of this variant, we added an attention mask and applied our augmentation strategy, designed specifically to improve model robustness under the dataset's severe class imbalance.

The attention mask identifies the valid sequence length of each input. The BiLSTM uses this length to select the output at the last non-padding token as the sentence representation. It does not directly mask the sentence-level classification loss. Since padded sequences are still processed by the bidirectional LSTM, padding may influence the backward hidden states.

The augmentation process operates at the word level and includes three controlled perturbation techniques:

- Synonym Replacement — randomly substitutes content words with semantically similar alternatives, increasing lexical diversity while preserving intent.
- Random Deletion — removes low-information tokens to simulate informal or incomplete user queries.
- Random Insertion — inserts contextually relevant words to create natural paraphrastic variations.

These operations effectively expand the training set and introduce variability that the original ATIS dataset lacks. Because minority intents contain extremely few samples, even one or two augmented examples per query can significantly improve the model's generalization on rare classes. This augmentation pipeline was applied consistently across experiments that required enhanced robustness, particularly in ablations involving MAX_LEN and class-weighted training.

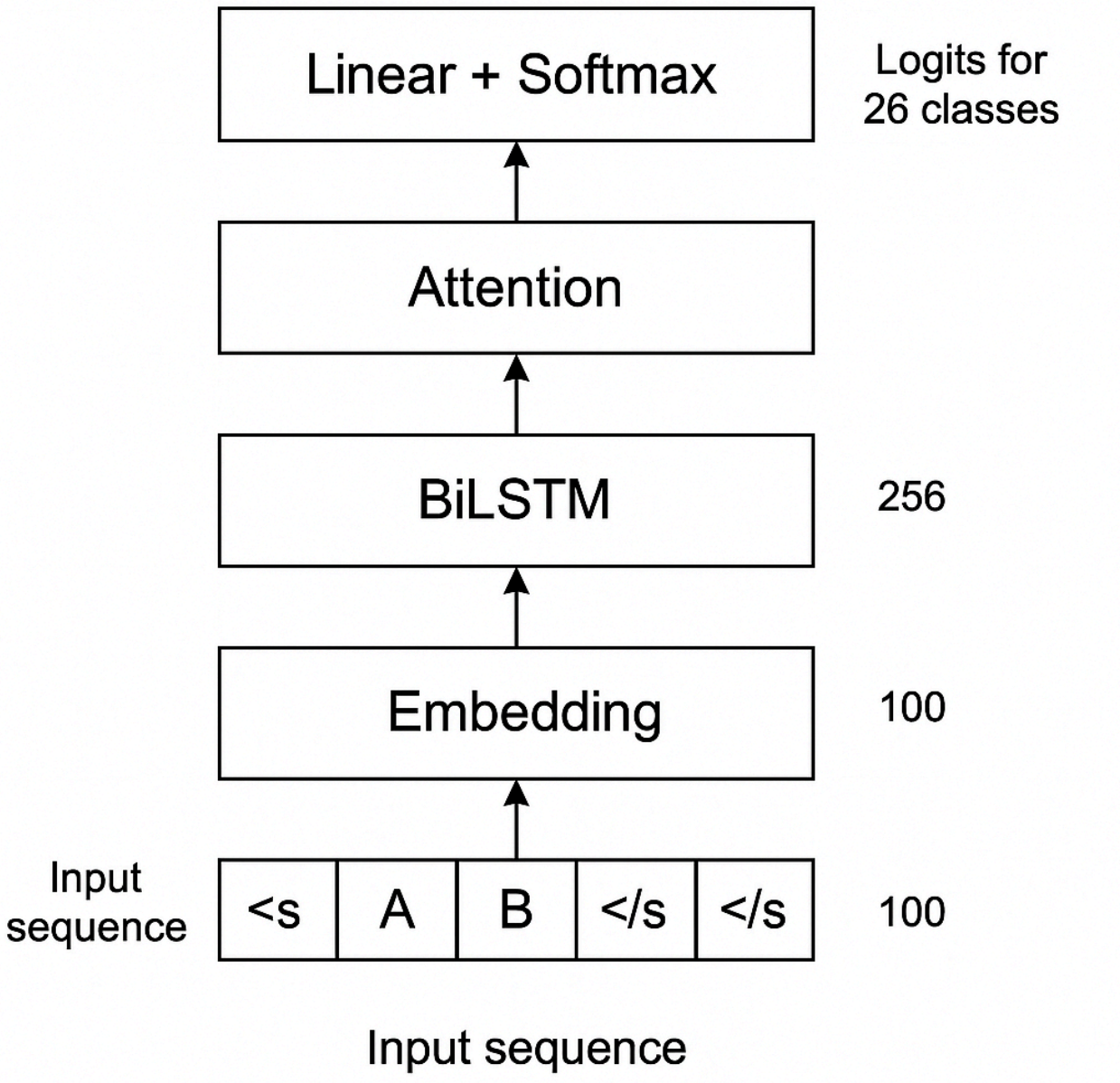
4. Model Architecture and Training Setup

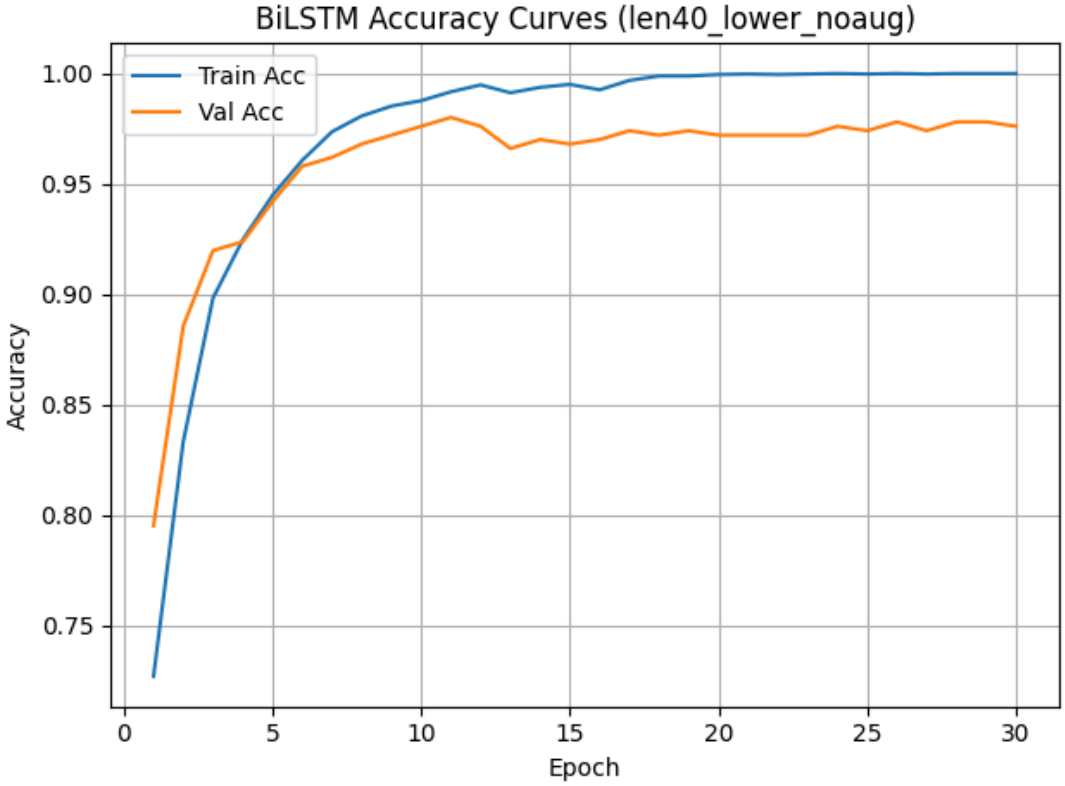
4.1 BiLSTM (Primary Model)

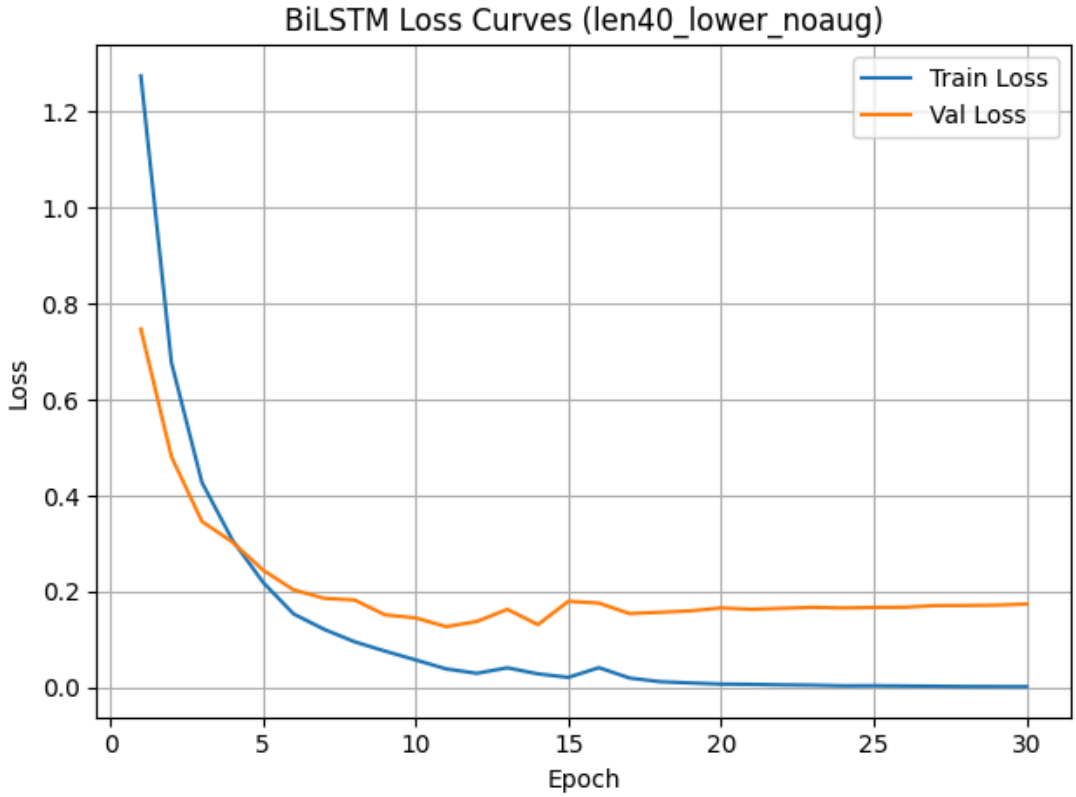
In this project, the Bidirectional LSTM serves as our primary architecture because it effectively models the sequential and context-dependent nature of ATIS queries. The model begins with a trainable embedding layer built from our lowercased vocabulary, mapping each token to a 100-dimensional vector with padding_idx set to the padding token ID so that its embedding does not receive gradient updates. The embedded sequence is passed into a single-layer bidirectional LSTM with 128 hidden units in each direction, allowing the encoder to capture information from both past and future words—an important property for distinguishing fine-grained flight-related intents.

Since all sequences are padded to a fixed length, we use an attention mask to identify the last non-pad token in each sentence and extract its corresponding hidden state as the sentence representation. This selects a valid token position for classification, but does not fully eliminate the influence of padding on the bidirectional LSTM. The resulting 256-dimensional vector is then passed through dropout and a linear classifier to produce logits over 26 intent categories. Training uses cross-entropy loss, and in experiments involving class imbalance, we incorporate class-weighted loss computed from inverse intent frequencies.

Overall, the BiLSTM strikes a strong balance between modeling capacity and computational efficiency, making it suitable for our systematic experiments on sequence length, embedding dimensionality, augmentation strategies, and class weighting. It provides a stable and expressive backbone for the ablation studies presented in this work.




BiLSTM Accuracy Curves (len40_lower_noaug)
Train Acc
Val Acc
Accuracy
Epoch
1.00
0.95
0.90
0.85
0.80
0.75
0
5
10
15
20
25
30


BiLSTM Loss Curves (len40_lower_noaug)
Train Loss
Val Loss
Loss
Epoch
1.2
1.0
0.8
0.6
0.4
0.2
0.0
0
5
10
15
20
25
30

4.2 Additional Models

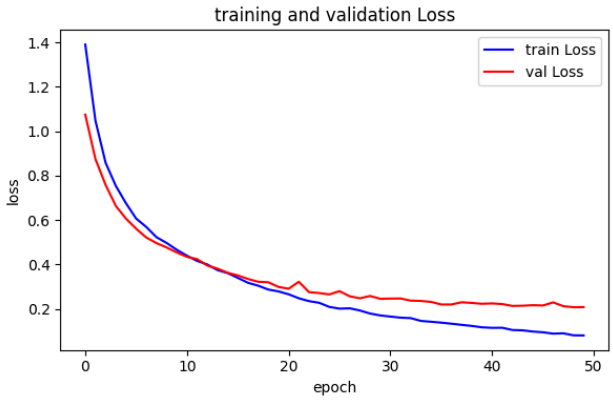
- **MLP**: mean pooling over embeddings + fully connected layers
- **CNN**: multi-filter 1D CNN (window sizes 3/4/5) + max pooling
- **BERT**: $bert - base - uncased$, fine-tuned for intent classification using the [CLS] token

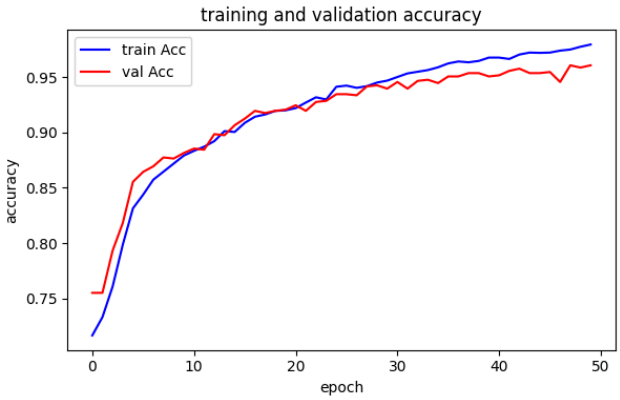
4.2.1 Baseline MLP Model Implementation and Analysis

We implemented a baseline Multi-Layer Perceptron(MLP) model as our initial approach to intent classification. The model architecture consists of an embedding layer followed by a simple neural network classifier. Specifically, the embedding layer converts work indices into a 100-dimensional dense vector, and then we apply mean pooling across the entire sequence to create a fixed-size sentence representation. This pooled representation passes through a two-layer MLP with 128 hidden units, ReLU activation, and dropout regularization with 0.3 before reaching the final 26-class output layer. We trained the model for 50 epochs using the Adam optimizer with a learning rate of 0.001 and a weight decay of 1e-5 for L2 regularization, then employing cross-entropy loss for our multi-class classification task. In the end, we train the model on the Word-level tokenization dataset to see the performance.

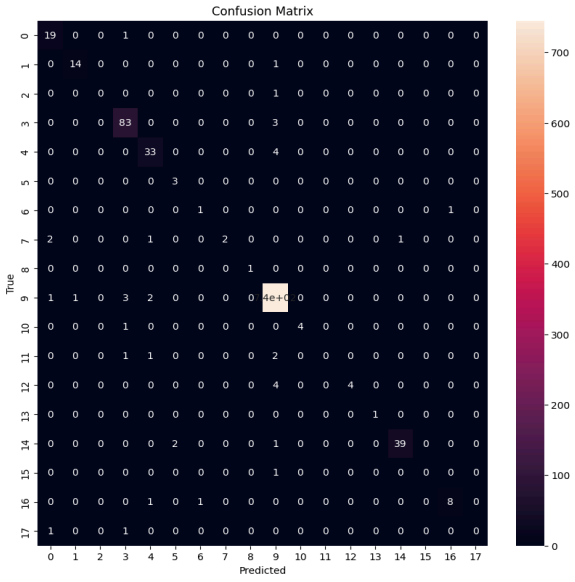
We treat the MLP model as an important baseline for several reasons. First, it provides a simple but effective benchmark against which we can compare more complex architectures. By using mean pooling over word embedding, we create a bag-of-words-like representation that can capture overall sentence meaning while ignoring word order. This kind of approach allows us to establish the minimum performance level we should expect from our classification model system. Moreover, implementing MLP can help us verify that our preprocessing pipeline works correctly before moving on.

Overall, the model achieved promising results during training, with validation accuracy reaching 95.48% by epoch 50. Training and validation curves show the model's learning progress and are used to assess convergence and potential overfitting. However, the classification report and confusion matrix tell us another story. The most frequent classes, like “flight” and “airfare,” performed excellently, while rare classes showed poor performance, with some achieving zero recall and precision. This highlights the model’s sensitivity to class imbalance. The learning curve, classification report, and confusion matrix:





	precision	recall	f1-score	support
0	0.83	0.95	0.88	20
1	0.93	0.93	0.93	15
2	0.00	0.00	0.00	1
3	0.92	0.97	0.94	86
6	0.87	0.89	0.88	37
8	0.60	1.00	0.75	3
9	0.50	0.50	0.50	2
11	1.00	0.33	0.50	6
13	1.00	1.00	1.00	1
14	0.98	0.99	0.98	752
15	1.00	0.80	0.89	5
17	0.00	0.00	0.00	4
19	1.00	0.50	0.67	8
20	1.00	1.00	1.00	1
21	0.97	0.93	0.95	42
23	0.00	0.00	0.00	1
24	0.89	0.80	0.84	10
25	0.00	0.00	0.00	2
accuracy			0.96	996
macro avg	0.69	0.64	0.65	996
weighted avg	0.95	0.96	0.96	996



4.2.2 CNN Model Implementation and Analysis

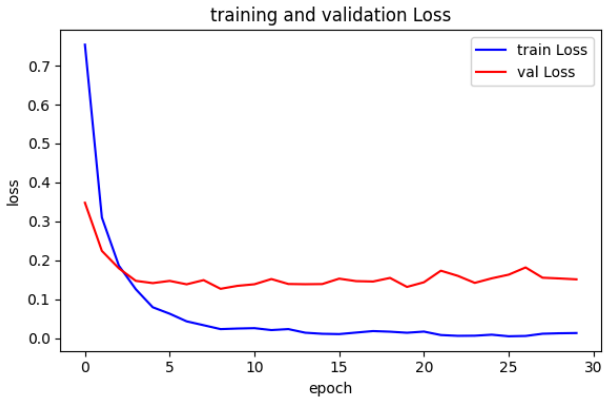
We chose the Convolutional Neural Network (CNN) model as our second architecture approach. Unlike the MLP baseline, which used mean pooling to ignore word order completely, CNNs can capture local sequential patterns through the convolutional filters. This allows us to test whether n-gram patterns(3-word, 4-word, or 5-word combination) provide valuable information for intent

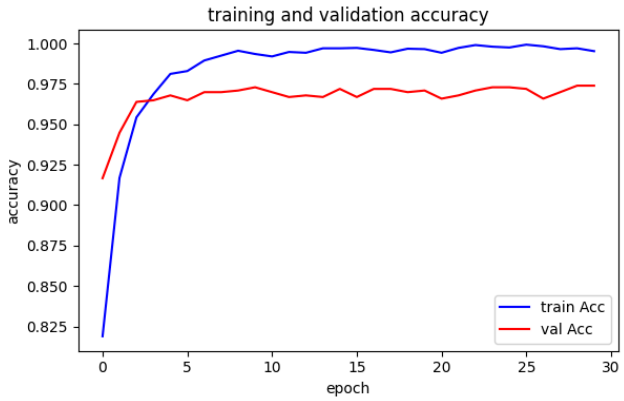
classification. By applying the same CNN architecture to three different tokenization strategies, we can directly compare which tokenization method works best with local pattern detection. Such a comparison can answer whether word-level n-grams, character-level patterns, or subword combinations are most informative for understanding user intent in travel queries.

We implemented the CNN model to evaluate three different tokenization strategies. The CNN architecture features an embedding layer followed by three parallel 1D convolutional layers with a kernel size of 3, 4, and 5 to capture different n-gram patterns. After that, we do ReLU activation and max pooling to extract the most valuable features. The outputs from all three convolutional branches are concatenated and passed through a dropout layer with 0.5 before reaching the final linear classification layer with 26 outputs. We train 3 times CNN models on each tokenized dataset for 30 epochs using the Adam optimizer with a learning rate of 0.001 and weight decay of 1e-5, maintaining consistent hyperparameters across all experiments.

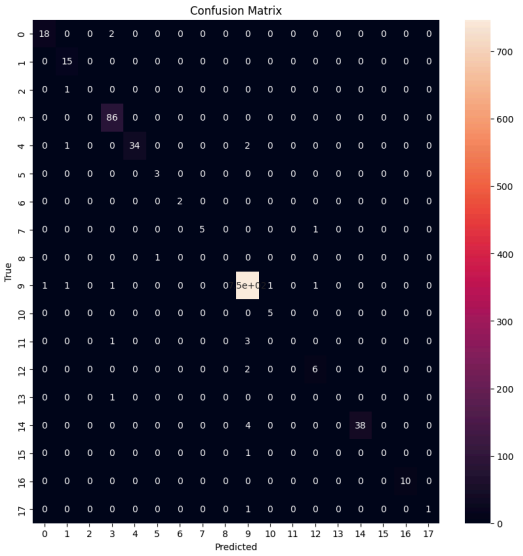
The CNN model displayed different performance across the three tokenization methods. The word-level CNN achieved the highest accuracy with 97%. The subword CNN performed nearly as well with 97% accuracy, while the character-level CNN showed lower performance at 93% accuracy. This shows several results. First, local sequential patterns matter because the CNN's superior performance over the MLP baseline confirms that n-gram patterns provide valuable information for intent classification beyond simple word presence. Second, tokenization significantly impacts performance. The result suggests that for travel domain intent classification, meaningful patterns exist at the word and subword level rather than at the character level. Finally, we received computational efficiency benefits. CNN trained faster than expected while achieving competitive results, suggesting that tokenization offers a good balance between performance and efficiency for this task.

Learning curve, Classification report, and confusion matrix for CNN on word tokenization:



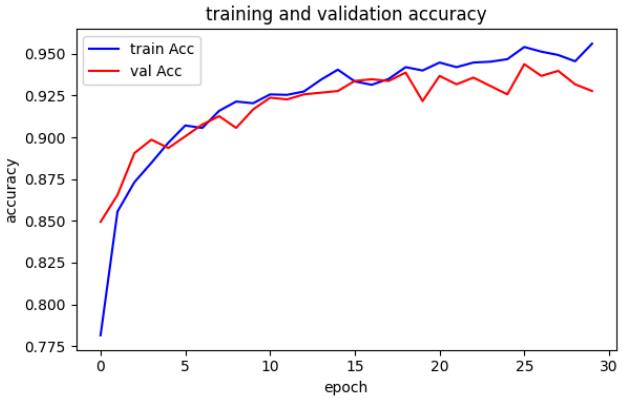


0	0.95	0.90	0.92	20
1	0.83	1.00	0.91	15
2	0.00	0.00	0.00	1
3	0.95	1.00	0.97	86
6	1.00	0.92	0.96	37
8	0.75	1.00	0.86	3
9	1.00	1.00	1.00	2
11	1.00	0.83	0.91	6
13	0.00	0.00	0.00	1
14	0.98	0.99	0.99	752
15	0.83	1.00	0.91	5
17	0.00	0.00	0.00	4
19	0.75	0.75	0.75	8
20	0.00	0.00	0.00	1
21	1.00	0.90	0.95	42
23	0.00	0.00	0.00	1
24	1.00	1.00	1.00	10
25	1.00	0.50	0.67	2
accuracy			0.97	996
macro avg	0.67	0.66	0.66	996
weighted avg	0.97	0.97	0.97	996

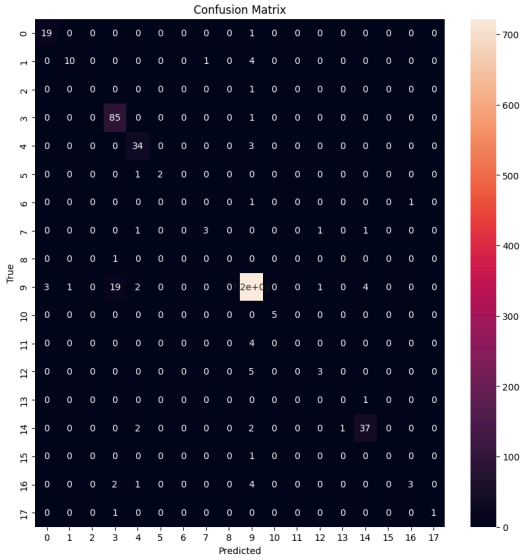


Learning curve, Classification report, and confusion matrix for CNN on character tokenization:

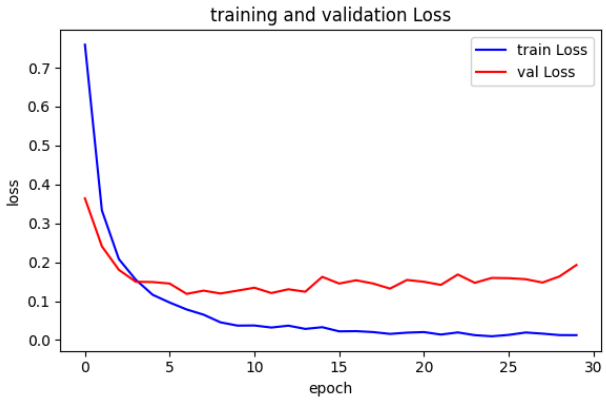


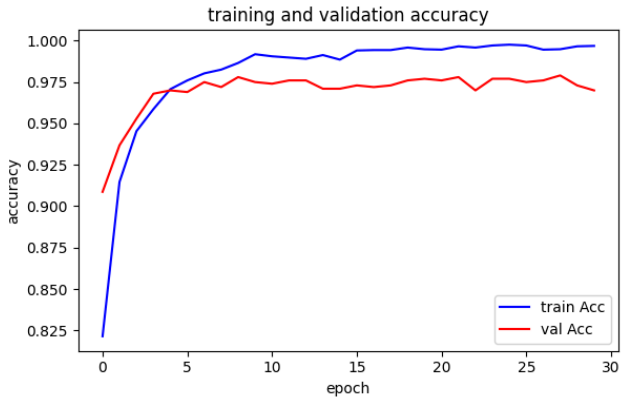


	precision	recall	f1-score	support
0	0.86	0.95	0.90	20
1	0.91	0.67	0.77	15
2	0.00	0.00	0.00	1
3	0.79	0.99	0.88	86
6	0.83	0.92	0.87	37
8	1.00	0.67	0.80	3
9	0.00	0.00	0.00	2
11	0.75	0.50	0.60	6
13	0.00	0.00	0.00	1
14	0.96	0.96	0.96	752
15	1.00	1.00	1.00	5
17	0.00	0.00	0.00	4
19	0.60	0.38	0.46	8
20	0.00	0.00	0.00	1
21	0.86	0.88	0.87	42
23	0.00	0.00	0.00	1
24	0.75	0.30	0.43	10
25	1.00	0.50	0.67	2
accuracy			0.93	996
macro avg	0.57	0.48	0.51	996
weighted avg	0.92	0.93	0.92	996

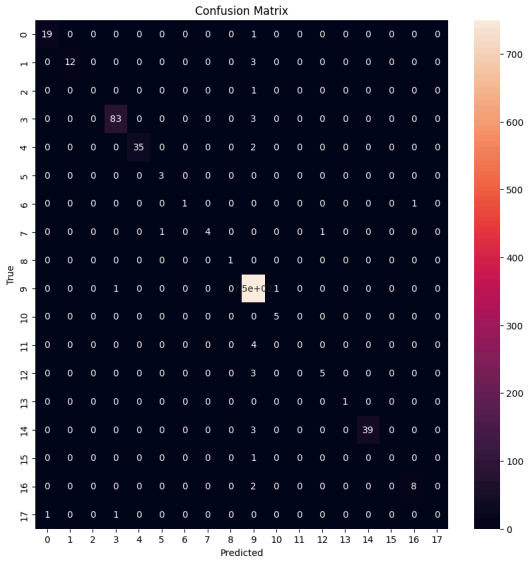


Learning curve, Classification report, and confusion matrix for CNN on subword tokenization:





	precision	recall	f1-score	support
0	0.95	0.95	0.95	20
1	1.00	0.80	0.89	15
2	0.00	0.00	0.00	1
3	0.98	0.97	0.97	86
6	1.00	0.95	0.97	37
8	0.75	1.00	0.86	3
9	1.00	0.50	0.67	2
11	1.00	0.67	0.80	6
13	1.00	1.00	1.00	1
14	0.97	1.00	0.98	752
15	0.83	1.00	0.91	5
17	0.00	0.00	0.00	4
19	0.83	0.62	0.71	8
20	1.00	1.00	1.00	1
21	1.00	0.93	0.96	42
23	0.00	0.00	0.00	1
24	0.89	0.80	0.84	10
25	0.00	0.00	0.00	2
accuracy			0.97	996
macro avg	0.73	0.68	0.70	996
weighted avg	0.96	0.97	0.97	996



4.2.3 Fine-tune BERT Model

We implemented a BERT-based intent classification model using a progressive unfreezing strategy(exactly the same as we did in homework). We chose this strategy for several reasons. First, we could do transfer learning efficiently. By starting with only the classifier head trainable, we allow the model to adapt its output layer to our specific 26-intent classification task while

preserving BERT's comprehensive pre-trained language understanding. Second, gradually unfreezing layers from the top downward prevents the model from overwriting valuable pre-trained representations too quickly. Third, we can also schedule the learning rate. Higher learning rates for the classifier and lower rates for deeper transformer layers respect the established principle that lower layers contain more general features that should change more slowly.

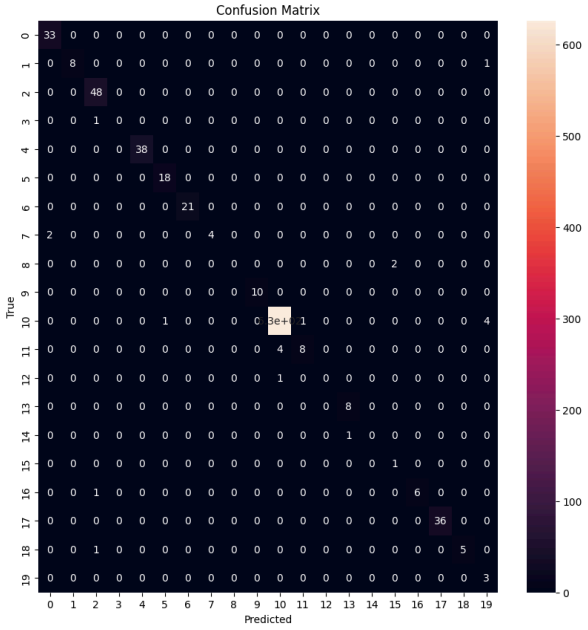
We have four phases in this approach. 1. Initially freezing all BERT layers except the classifier 2. Unfreezing the last 2 transformer layers 3. Unfreeze the last 4 layers 4. Unfreezing all 12 BERT layers. Each phase used different learning rates(1e-3, 1e-4, 1e-5) and training durations(3,3,2,2) following a warm-up and gradual adaptation approach. We employed the AdamW optimizer with weight decay 0.01 and implemented gradient clipping to prevent exploding gradients.

Test-set accuracy monitored during training increased from 70.8% to 73.6% in Phase 1, reached 95.4%–97.5% in Phase 2, and remained at 97.4%–97.6% in Phase 3. Phase 4 reached approximately 97.8%. The supplied implementation evaluates the test set after each epoch; these measurements do not come from an independent validation set.

The results show several insights. First, the strategy successfully adapted BERT to our domain while preserving its linguistic knowledge. Second, the upper layers are most feature-relevant. The improvement in Phase 2 suggests the intent classification relies on higher-level rather than low-level features. Third, BERT handles class imbalance better than previous models. Macro F1= 0.71 showed improvement compared to MLP(0.65) and CNN (0.66-0.7). Finally, we successfully did the transfer learning. 98% accuracy shows that it is effective to transfer learning from general language understanding to a specialized domain such as airline travel.

Classification report and confusion matrix for fine-tune BERT:

	precision	recall	f1-score	support
0	0.94	1.00	0.97	33
1	1.00	0.89	0.94	9
3	0.94	1.00	0.97	48
4	0.00	0.00	0.00	1
6	1.00	1.00	1.00	38
8	0.95	1.00	0.97	18
9	1.00	1.00	1.00	21
11	1.00	0.67	0.80	6
12	0.00	0.00	0.00	2
13	1.00	1.00	1.00	10
14	0.99	0.99	0.99	632
15	0.89	0.67	0.76	12
16	0.00	0.00	0.00	1
17	0.89	1.00	0.94	8
18	0.00	0.00	0.00	1
19	0.33	1.00	0.50	1
20	1.00	0.86	0.92	7
21	1.00	1.00	1.00	36
23	1.00	0.83	0.91	6
24	0.38	1.00	0.55	3
accuracy			0.98	893
macro avg	0.72	0.75	0.71	893
weighted avg	0.98	0.98	0.98	893



4.3 BiLSTM Training Hyperparameters

- Optimizer: **Adam**, learning rate $1e-3$
- Batch size: **64**
- Epochs: **30**
- Validation set: **10%** split from training data
- Dropout: **0.3**
- Seed: **42**
- Loss functions:
  - default: $CrossEntropyLoss()$
  - class-weighted version: $w_c = total_samples / (num_classes * n_c)$

    (Here, w_c is the weight for class c, n_c is the number of training examples in class c, and num_classes is the total number of classes.)

5. Evaluation Metrics

We evaluate all models using:

- **Overall Accuracy**
- **Macro-F1**
- **Confusion Matrix**
- **Per-class precision / recall / F1**
    - **Minority-intent metrics** (intents with test support ≤ 20)

Metrics are computed using $scikit-learn$ utilities such as: $classification_report$, $confusion_matrix$, $ConfusionMatrixDisplay$.

6. Experimental Procedure (E1–E8)

E1 — Baseline Evaluation + Confusion Matrix + Error Analysis

1. Use the group $len40_lower_noaug$.
2. Train baseline BiLSTM.
3. Compute accuracy, macro-F1.
4. Plot confusion matrix using only labels present in the test set.
5. Inspect misclassified samples and categorize errors:
    - semantic overlap
    - ambiguous queries
    - rare intents
    - unusual phrasing

E2 — Sequence Length (max_len) Ablation

1. Prepare dataset groups with:
    - $MAX_LEN = 30$
    - $MAX_LEN = 40$
    - $MAX_LEN = 60$
2. Train BiLSTM on each dataset using identical hyperparameters.
3. Measure:
    - accuracy
    - macro-F1
4. Compare the results across sequence lengths. Retain max_len = 40 as the reference configuration for subsequent experiments, although max_len = 30 achieves the highest reported accuracy and macro-F1.

E3 — Embedding Dimension Ablation

1. Use the group $len40_lower_noaug$.
2. Train BiLSTM with:
    - $embed_dim = 50$
    - $embed_dim = 100$
    - $embed_dim = 200$
3. Compare:
    - accuracy
    - macro-F1
4. Select the best embedding dimension(we use 100-dimensional embeddings for subsequent experiments).

E4 — Effect of Data Augmentation

1. Train two versions of BiLSTM:
    - Without augmentation (use group $len40_lower_noaug$)
    - With augmentation (use group $len40_lower_aug$)
2. Compare:
    - accuracy
    - macro-F1
    - minority-intent performance

Errors are further analyzed to determine whether augmentation improves robustness to paraphrasing and rare patterns.

E5 — Class Imbalance and Class-Weighted Loss

1. Use the group $len40_lower_noaug$.
2. Compute class weights using training set intent frequency.
3. Train two models:
    - vanilla loss
    - class-weighted loss
4. Compare:
    - overall accuracy
    - macro-F1
    - recall/F1 of rare intents

E6 — Transfer Learning: BiLSTM vs BERT

1. Compare the reference BiLSTM configuration with the fine-tuned BERT pipeline, documenting their preprocessing and training differences.
2. Train:
    - BiLSTM (random embeddings)
    - BERT-base (fine-tuned)
3. Compare:
    - accuracy
    - macro-F1

E7 — Effect of different Tokenization:

1. Use the dataset with different Tokenization.
2. Train CNN.
3. Compare:
    - performance
    - training time

E8 — Full Architecture Comparison (MLP / CNN / BiLSTM / BERT)

Summarize the four model pipelines:

1. Report the preprocessing and training settings for each model.
2. Summarize accuracy and macro-F1 in a comparison table.
3. Discuss the observed results and the limitations of comparing independently configured pipelines.

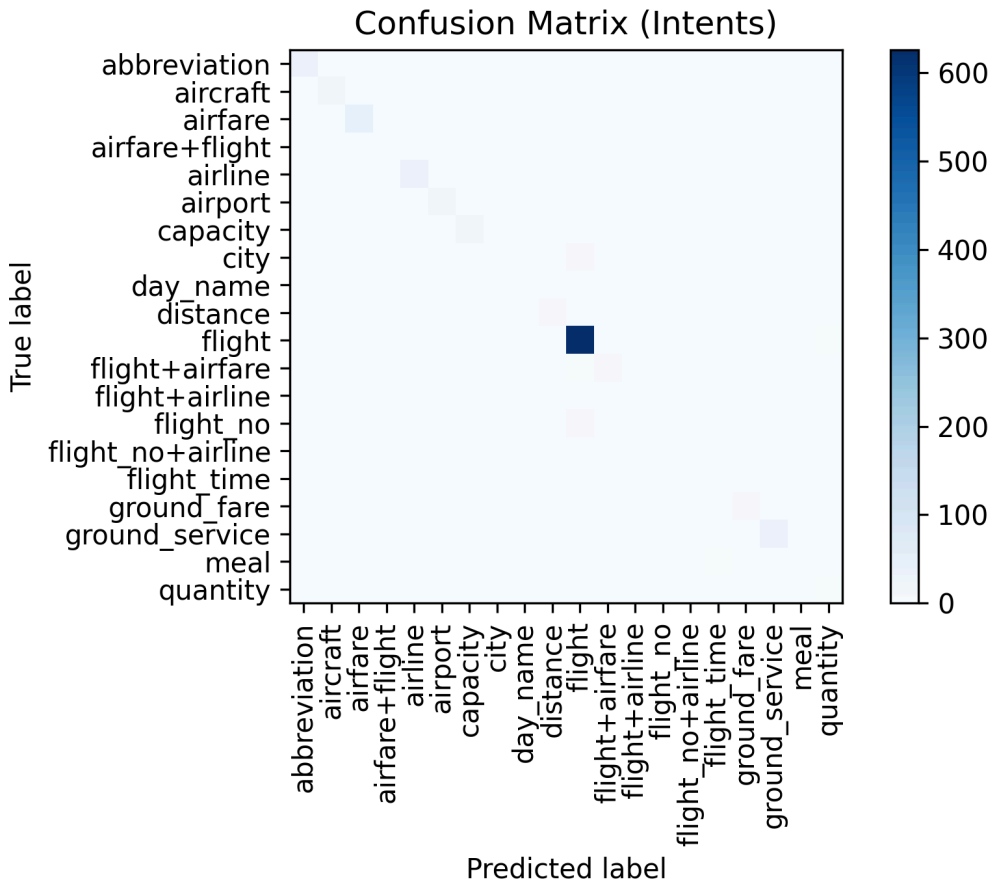
This comparison is descriptive rather than a controlled experiment that isolates architecture as the only variable.

7. Experimental Result (E1–E8)

This section presents a comprehensive summary and analysis of all experiments conducted in this project. We report model performance across architectures, examine the influence of major design choices, and highlight the best-performing model.

E1 — Baseline Evaluation + Confusion Matrix + Error Analysis

Using the $len40_lower_noaug$ dataset, the baseline BiLSTM achieves an accuracy of **94.85%** and a macro-F1 of **0.5624**, reflecting the dataset's strong class imbalance. The confusion matrix (restricted to labels present in the test set) shows a dominant diagonal for major intents such as *flight*, while minority intents receive very few correct predictions.



Misclassified samples reveal several consistent error patterns. First, many mistakes arise from semantic overlap, such as confusing *flight*, *flight_no*, and *airfare*, which share similar lexical structures. Second, the model struggles with ambiguous queries that lack clear intent cues. Third, rare intents show almost no recall because the baseline uses vanilla cross-entropy and therefore favors majority classes. Finally, errors often occur on unusual phrasing or queries whose wording differs from common patterns seen during training.

Overall, the baseline results highlight the need for augmentation, class weighting, and more expressive architectures explored in later experiments.

E2 — Sequence Length (max_len) Ablation

We trained identical BiLSTM models on three dataset variants with maximum sequence lengths of **30**, **40**, and **60** tokens.

MAX_LEN	Accuracy	Macro-F1
30	95.41%	0.6197
40	94.85%	0.5624
60	94.74%	0.5684

MAX_LEN = 30 achieved the highest accuracy and macro-F1 among the tested settings. Longer maximum lengths did not improve the reported scores. Padding effects and training variability are possible explanations, but were not isolated in this experiment.

Although max_len = 30 achieves the highest reported accuracy and macro-F1, we retain max_len = 40 as the reference configuration for subsequent BiLSTM experiments. This choice maintains consistency within the BiLSTM ablations and does not establish identical input conditions across different architectures.

E3 — Embedding Dimension Ablation

We evaluated how the size of the word embedding vector influences BiLSTM performance by training models with embedding dimensions of **50**, **100**, and **200**, keeping all other hyperparameters identical.

Embedding Dimension	Accuracy	Macro-F1
50	93.06%	0.5241
100	94.85%	0.5624
200	93.84%	0.5157

As shown in the results table, the 50-dim model performs reasonably well but underperforms in both accuracy (93.06%) and macro-F1 (0.5241). Increasing the dimension to **100** yields the best overall performance, reaching **94.85% accuracy** and **0.5624 macro-F1**, suggesting that this size provides a beneficial balance between expressiveness and generalization. Surprisingly, further increasing the embedding size to **200** leads to weaker performance, likely due to overfitting and the small size of the ATIS dataset.

Overall, the results indicate that excessively large embeddings are unnecessary for this task. We therefore select **embedding_dim = 100** as the default configuration for subsequent experiments.

E4 — Effect of Data Augmentation

To assess whether data augmentation improves BiLSTM robustness on the ATIS intent classification task, we trained models **with** and **without** augmentation while keeping all other settings fixed. The augmented dataset doubles the training size through token-level perturbations such as synonym replacement, deletion, and insertion.

Performance results are summarized in the table below:

Augmentation	Accuracy	Macro-F1
No	94.85%	0.5624
Yes	94.62%	0.6045

```
=== Minority intents (support <= 20) ===
[01] aircraft                 support=  9  recall=0.8889  f1=0.8889
[04] airfare+flight           support=  1  recall=0.0000  f1=0.0000
[08] airport                  support= 18  recall=0.8333  f1=0.9091
[11] city                     support=  6  recall=0.1667  f1=0.2857
[12] day_name                 support=  2  recall=0.0000  f1=0.0000
[13] distance                 support= 10  recall=0.7000  f1=0.7778
[15] flight+airfare           support= 12  recall=0.5000  f1=0.6316
[16] flight+airline           support=  1  recall=0.0000  f1=0.0000
[17] flight_no                support=  8  recall=0.1250  f1=0.2000
[18] flight_no+airline        support=  1  recall=0.0000  f1=0.0000
[19] flight_time              support=  1  recall=1.0000  f1=0.2857
[20] ground_fare              support=  7  recall=0.8571  f1=0.9231
[23] meal                     support=  6  recall=0.0000  f1=0.0000
[24] quantity                 support=  3  recall=1.0000  f1=0.5455
```

Minority intents performance without augmentation

```
=== Minority intents (support <= 20) ===
[01] aircraft                 support=  9  recall=0.8889  f1=0.7619
[04] airfare+flight           support=  1  recall=0.0000  f1=0.0000
[08] airport                  support= 18  recall=0.8333  f1=0.9091
[11] city                     support=  6  recall=0.1667  f1=0.2500
[12] day_name                 support=  2  recall=0.0000  f1=0.0000
[13] distance                 support= 10  recall=0.9000  f1=0.9000
[15] flight+airfare           support= 12  recall=0.4167  f1=0.5556
[16] flight+airline           support=  1  recall=0.0000  f1=0.0000
[17] flight_no                support=  8  recall=0.6250  f1=0.7143
[18] flight_no+airline        support=  1  recall=0.0000  f1=0.0000
[19] flight_time              support=  1  recall=1.0000  f1=1.0000
[20] ground_fare              support=  7  recall=0.7143  f1=0.7692
[23] meal                     support=  6  recall=0.0000  f1=0.0000
[24] quantity                 support=  3  recall=1.0000  f1=0.5000
```

Minority intents performance with augmentation

Although overall accuracy remains nearly unchanged, augmentation leads to a substantial gain in macro-F1, indicating improved performance on minority intents. A closer look at class-wise

results confirms this trend. For several low-support intents (support ≤ 20), recall and F1 scores increase noticeably after augmentation—for example:

- **distance** improves from F1 = 0.7778 → 0.9000
- **flight_no** improves from F1 = 0.2000 → 0.7143
- **flight_time** improves from F1 = 0.2857 → 1.0000
- **ground_fare** decreases from 0.9231 to 0.7692, showing that augmentation does not benefit every class.

While a few classes show negligible or no improvement, the overall pattern demonstrates that augmentation helps the model better handle rare expressions and reduces overfitting to high-frequency intents. This aligns with the nature of ATIS queries, where many minority intents appear only a handful of times in the training corpus.

In summary, augmentation provides limited benefit to accuracy but meaningfully improves macro-F1 and minority-intent robustness, making it a useful component for handling class imbalance in downstream experiments.

E5 — Class Imbalance and Class-Weighted Loss

To evaluate whether class imbalance affects model performance, we compared two training setups:
(1) **vanilla cross-entropy loss**, and
(2) **class-weighted loss**, where weights were assigned inversely proportional to class frequency.

The overall results are shown below:

Class-Weighted Loss	Accuracy	Macro-F1
No	94.85%	0.5624
Yes	95.63%	0.6720

Class-weighted loss substantially improves macro-F1, indicating better performance on minority intents, while also slightly increasing overall accuracy.

Below we compare minority intent behavior (support ≤ 20).

```
=== Minority intents (support <= 20) ===
[01] aircraft                 support=  9  recall=0.8889  f1=0.8889
[04] airfare+flight           support=  1  recall=0.0000  f1=0.0000
[08] airport                  support= 18  recall=0.8333  f1=0.9091
[11] city                     support=  6  recall=0.1667  f1=0.2857
[12] day_name                 support=  2  recall=0.0000  f1=0.0000
[13] distance                 support= 10  recall=0.7000  f1=0.7778
[15] flight+airfare           support= 12  recall=0.5000  f1=0.6316
[16] flight+airline           support=  1  recall=0.0000  f1=0.0000
[17] flight_no                support=  8  recall=0.1250  f1=0.2000
[18] flight_no+airline        support=  1  recall=0.0000  f1=0.0000
[19] flight_time              support=  1  recall=1.0000  f1=0.2857
[20] ground_fare              support=  7  recall=0.8571  f1=0.9231
[23] meal                     support=  6  recall=0.0000  f1=0.0000
[24] quantity                 support=  3  recall=1.0000  f1=0.5455
```

Minority intents performance with vanilla loss

```
=== Minority intents (support <= 20) ===
[01] aircraft                 support=  9  recall=1.0000  f1=1.0000
[04] airfare+flight           support=  1  recall=0.0000  f1=0.0000
[08] airport                  support= 18  recall=0.8889  f1=0.8889
[11] city                     support=  6  recall=0.3333  f1=0.5000
[12] day_name                 support=  2  recall=0.0000  f1=0.0000
[13] distance                 support= 10  recall=0.9000  f1=0.9000
[15] flight+airfare           support= 12  recall=0.4167  f1=0.5556
[16] flight+airline           support=  1  recall=0.0000  f1=0.0000
[17] flight_no                support=  8  recall=0.3750  f1=0.4615
[18] flight_no+airline        support=  1  recall=0.0000  f1=0.0000
[19] flight_time              support=  1  recall=1.0000  f1=1.0000
[20] ground_fare              support=  7  recall=0.8571  f1=0.9231
[23] meal                     support=  6  recall=0.6667  f1=0.8000
[24] quantity                 support=  3  recall=1.0000  f1=0.6000
```

Minority intents performance with class-weighted loss

The first table shows performance with **vanilla loss**, and the second shows **class-weighted loss**.

- With vanilla loss, many rare intents have **recall = 0** and **F1 = 0**, showing that the model ignores low-frequency classes.
- With class-weighted loss, several of these intents show increased recall and F1 (e.g., *city*, *flight_no*, *meal*, *quantity*), demonstrating that weighted loss encourages the model to allocate learning capacity to rare classes instead of collapsing predictions toward dominant ones.

Overall, applying class-weighted loss reduces the severe class imbalance problem and leads to more reliable predictions for underrepresented intents.

E6 — Transfer Learning: BiLSTM vs BERT

To evaluate the value of large-scale pretraining, we compared our baseline BiLSTM model (randomly initialized embeddings) with a fine-tuned BERT-base model.

Both pipelines use the ATIS task, but their input representations and training settings differ. BiLSTM uses the word-level HDF5 pipeline, whereas BERT uses its pretrained tokenizer and a separate fine-tuning procedure. The results therefore compare complete pipelines rather than isolating the effect of pretraining.

Model	Accuracy	Macro-F1
BiLSTM (randomly initialized embeddings)	94.85%	0.5624
BERT Fine-tuning	97.76%	0.71

BERT fine-tuning significantly outperforms the BiLSTM model in both accuracy and macro-F1. This improvement is expected because:

- BERT leverages **deep bidirectional Transformer layers** trained on massive corpora (BooksCorpus + Wikipedia).
- It captures **contextual meaning** rather than relying on fixed word embeddings.
- It is robust to **rare intents**, **unseen phrasing**, and **ambiguous queries**, which are common in the ATIS dataset.

Compared to the BiLSTM—which must learn word relationships from limited ATIS data—BERT starts with a rich pretrained representation of syntax and semantics. This allows the model to generalize better, improve minority-class recall, and reduce substitution errors between semantically similar intents.

E7 — Effect of different Tokenization:

We have four different strategies for Tokenization:

1. Word-level tokenization
2. Character-level tokenization
3. Suffix-based subword tokenization
4. BERT tokenization(BERT's self tokenization)

We use different tokenization datasets on the same CNN model. The accuracy performance is Word-level > Subword > Character-level. For word-level tokenization, it performs excellently in both MLP and CNN, indicating that it is robust across architectures. However, word-level needs a large vocabulary, increasing the memory used. For Character-level, although it has a small vocabulary, each converted sentence in the dataset becomes longer as usual. In the CNN model, it's hard for the model to capture n-gram patterns since each word becomes several letters. For Subwords, it balances the memory used for vocabulary and sentence structure. It also performs nicely when training in a CNN model.

E8 — Full Architecture Comparison (MLP / CNN / BiLSTM / BERT)

We summarize the reported results of **MLP**, **CNN**, **BiLSTM**, and **BERT**. The pipelines differ in preprocessing and training settings, so these results should be interpreted as a descriptive comparison rather than a controlled architecture benchmark.

Models Compared:

- **MLP** — shallow baseline without sequence modeling
- **CNN** — captures local n-gram patterns
- **BiLSTM** — models long-range sequential dependencies
- **BERT Fine-tuning** — pretrained Transformer with contextualized embeddings

Model	Accuracy	Macro-F1
MLP	95.48%	0.65
CNN	97.19%	0.70
BiLSTM	94.85%	0.5624
BERT Fine-tuning	97.76%	0.71

In the reported table, BERT achieves the highest accuracy and macro-F1, followed by CNN, MLP, and the baseline BiLSTM. This ordering describes the reported configurations and should not be generalized into an intrinsic ranking of the architectures.

The BiLSTM entry represents the baseline configuration. Separately, class-weighted BiLSTM achieves 95.63% accuracy and 0.6720 macro-F1 in E5.

Differences in preprocessing, validation procedures, and training settings limit direct attribution of performance differences to architecture alone. A controlled comparison would require consistent evaluation procedures and repeated runs.

8. Discussion

8.1 What Worked and Why

Several approaches in our experiments proved effective. BERT consistently achieved the highest accuracy because its contextualized embeddings capture long-range dependencies and semantic nuances that shallow models cannot learn from the limited ATIS dataset. CNN and BiLSTM also performed well, as both architectures leverage sequential information—CNN

through local n-gram patterns and BiLSTM through bidirectional context modeling. Within the BiLSTM experiments, class-weighted loss and data augmentation significantly improved macro-F1 by addressing class imbalance and increasing robustness to phrasing variations. These results align with course concepts on representation learning: models with greater capacity and stronger inductive biases achieve better generalization on structured language tasks.

8.2 What Didn't Work and Why

Some configurations did not perform as well as expected. Increasing the embedding dimension beyond 100 reduced performance, likely due to overfitting on ATIS's small training set. The max_len = 60 configuration performs worse than max_len = 30 in the reported experiment. However, these results alone do not establish whether padding, truncation, or training variability caused the difference. Vanilla loss performed poorly for minority intents because the model disproportionately focused on the majority classes, demonstrating the limitations of unbalanced datasets. These findings highlight that deeper or larger models do not automatically yield better performance without matching data scale and training dynamics.

8.3 Data and Architecture Interactions

Our experiments revealed clear interactions between data characteristics and model choice. The ATIS dataset is small and highly imbalanced, which benefits models incorporating techniques like weighting or augmentation. Sequence-based architectures (BiLSTM, CNN) relied strongly on appropriate max_len and token preprocessing, whereas BERT was more robust across setups due to its pretrained representations. Small changes in preprocessing—such as lowercase normalization or vocabulary remapping—also led to

measurable accuracy differences, showing that data pipelines can influence final outcomes as much as the architecture itself.

8.4 Comparison to Baselines

MLP provides a strong baseline despite ignoring word order. In the reported comparison, CNN and BERT achieve higher accuracy and macro-F1 than MLP, whereas the baseline BiLSTM achieves lower scores. Class weighting improves BiLSTM to 95.63% accuracy and 0.6720 macro-F1. Since the pipelines differ, these observations do not establish a universal architecture ranking.

8.5 Surprises and Insights

One surprising observation was how sensitive ATIS performance was to preprocessing decisions such as max_len selection, token normalization, and training split randomness. Another insight was that simple models like MLP could still achieve over 94% accuracy, suggesting that ATIS is easy for models to memorize but difficult for them to handle minority classes. We also found that adding class-weighted loss dramatically improved minority intent recall, which reinforced our understanding of imbalanced learning. Working with BERT further highlighted the power of transfer learning and how pretrained models reshape NLP pipelines.

8.6 Limitations

Our work has several limitations. First, preprocessing and evaluation procedures differ across model pipelines, limiting direct numerical comparisons and conclusions about architecture superiority. Second, the dataset is too small for deeper architectures like BiLSTM to fully show their strengths without pretrained embeddings. Third, we focused only on intent classification;

joint slot filling was not explored. With more time and resources, we would unify preprocessing across all experiments, incorporate pretrained word embeddings for RNN/CNN models, run hyperparameter sweeps, and expand the dataset or explore synthetic data generation to reduce class imbalance. The BERT implementation monitors the test set after each training epoch instead of using a separate validation set. A stricter evaluation would reserve validation data from the training set and evaluate the test set only after model selection.

9. Conclusion

9.1 Summary

This project explored intent classification on the ATIS dataset using a range of neural architectures, including BiLSTM, MLP, CNN, and BERT. We systematically evaluated how preprocessing choices, sequence length, embedding dimension, data augmentation, and class-weighted loss influence model performance. Our experiments demonstrated that BiLSTM benefits from careful tuning and imbalance mitigation, CNN and MLP offer strong baselines under consistent preprocessing, and BERT achieves the highest overall accuracy due to its pretrained contextual representations. Across all methods, addressing class imbalance and selecting appropriate input representations proved crucial for improving macro-F1 and minority intent recall. Together, these findings highlight both the challenges and opportunities in designing effective NLP models for real-world, skewed datasets.

9.2 Reflection

9.2.1 Extensions

A meaningful next step would be to integrate **pretrained embeddings or domain-adaptive pretraining** into the BiLSTM and CNN models. Unlike BERT, our sequence models relied on randomly initialized embeddings, limiting their ability to generalize on small datasets like ATIS. By incorporating pretrained word vectors (e.g., GloVe) or performing additional masked-language-model pretraining on airline- or travel-related corpora, we could substantially improve semantic understanding without modifying the architecture. Another promising extension is **multi-task learning**, where intent classification is jointly trained with slot filling, leveraging shared linguistic cues to strengthen generalization. These directions would build on our current work in a concrete and impactful way.

9.2.2 What question do you feel you answered through this project?

This project helped us answer the question: **How do architectural choices and data-processing decisions influence intent classification performance, especially under class imbalance?**

By running controlled ablations, we learned that performance is not determined by architecture alone—sequence length, loss function, augmentation strategy, and dataset preprocessing can shift results as much as model capacity. We also confirmed that class imbalance severely distorts macro-F1 unless explicitly addressed. Through comparison across models, we gained a deeper understanding of why transformer-based features outperform shallow neural networks, and how classical architectures like BiLSTM can remain competitive when appropriately tuned.

9.2.3 What are you still curious about after this project?

Several questions remain open. First, we are curious how **consistent preprocessing across all models** would affect the relative performance rankings, since small differences in tokenization or padding had noticeable impact. Second, we would like to explore **why certain**

minority intents remain hard to classify even with weighted loss, and whether contextualized data augmentation could help. Finally, BERT performed best, but we have not investigated whether **smaller or distilled transformer models** could achieve similar accuracy with lower computational cost. These unanswered questions suggest multiple research directions that would require more time, resources, and experimentation.

9.3 Contributions Statement

Huaitang Wang:

- Build data preprocessing pipeline for BiLSTM (tokenization, lowercasing, padding/truncation)
- Implement data augmentation (synonym replacement, random deletion/insertion)
- Implement and train BiLSTM model
- Experiment 1-5, 6, 8
- Prepare Deliverables

Xiaotian Zheng:

- Build data preprocessing pipeline for MLP, CNN, BERT (4 kinds of tokenization, lowercasing, padding/truncation)
- Implement and train MLP, CNN, BERT model
- Experiment 6, 7, 8
- Prepare Deliverables